

END TO END DATA CLEANING

GUIBERT Thomas

&

HAMILTON Djibril

18/04/2025

OBJECTIVES

1.Exploring the data.

Python request:

```
Entrée [1]: 1 import pandas as pd
2
3 # Load the dataset
4 df = pd.read_csv('credit_data.csv')
5
6 # Display basic info
7 print(df.info())
8 print(df.head())
9
10 # Check target distribution
11 print(df['Risk'].value_counts())
```

Result:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            1000 non-null  int64
1   Age                   1000 non-null  int64
2   Sex                   1000 non-null  object
3   Job                   1000 non-null  int64
4   Housing               1000 non-null  object
5   Saving accounts       817 non-null   object
6   Checking account      606 non-null   object
7   Credit amount         1000 non-null  int64
8   Duration              1000 non-null  int64
9   Purpose               1000 non-null  object
10  Risk                  1000 non-null  object
dtypes: int64(5), object(6)
memory usage: 86.1+ KB
None
   Unnamed: 0  Age  Sex  Job  Housing  Saving accounts  Checking account \
0           0   67  male    2    own             NaN         little
1           1   22  female  2    own             little      moderate
2           2   49  male    1    own             little             NaN
3           3   45  male    2    free             little         little
4           4   53  male    2    free             little         little

   Credit amount  Duration  Purpose  Risk
0          1169         6      radio/TV  good
1          5951        48      radio/TV  bad
2          2096        12    education  good
3          7882        42  furniture/equipment  good
4          4870        24           car  bad

Risk
good    700
bad     300
Name: count, dtype: int64
```

Python request:

```
Entrée [2]: 1 print(df.describe())
2
```

Result:

	Unnamed: 0	Age	Job	Credit amount	Duration
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	499.500000	35.546000	1.904000	3271.258000	20.903000
std	288.819436	11.375469	0.653614	2822.736876	12.058814
min	0.000000	19.000000	0.000000	250.000000	4.000000
25%	249.750000	27.000000	2.000000	1365.500000	12.000000
50%	499.500000	33.000000	2.000000	2319.500000	18.000000
75%	749.250000	42.000000	2.000000	3972.250000	24.000000
max	999.000000	75.000000	3.000000	18424.000000	72.000000

Python request:

```
Entrée [3]: 1 # Function to analyze a column
2 def analyze_column(column_name):
3     print(f"\nAnalysis for '{column_name}':")
4     print("-" * 30)
5
6     # Count of missing values
7     missing = df[column_name].isna().sum()
8     print(f"Missing values: {missing} ({missing/len(df)*100:.1f}%)")
9
10    # Value counts
11    value_counts = df[column_name].value_counts(dropna=False)
12    print("\nValue counts:")
13    print(value_counts)
14
15    # Most frequent value
16    most_frequent = df[column_name].mode()[0]
17    print(f"\nMost frequent value: '{most_frequent}'")
18
19    # Visualization
20    df[column_name].value_counts(dropna=False).plot(kind='bar', title=f'Distribution of {column_name}')
21
22    return most_frequent
23
24    # Analyze Saving accounts
25    saving_most_frequent = analyze_column('Saving accounts')
26
27    # Analyze Checking account
28    checking_most_frequent = analyze_column('Checking account')
29
30    print("\nRecommended replacement values:")
31    print(f"- For 'Saving accounts': '{saving_most_frequent}'")
32    print(f"- For 'Checking account': '{checking_most_frequent}'")
```

Result:

```

Analysis for 'Saving accounts':
-----
Missing values: 183 (18.3%)

Value counts:
Saving accounts
little      603
NaN         183
moderate    103
quite rich   63
rich        48
Name: count, dtype: int64

Most frequent value: 'little'

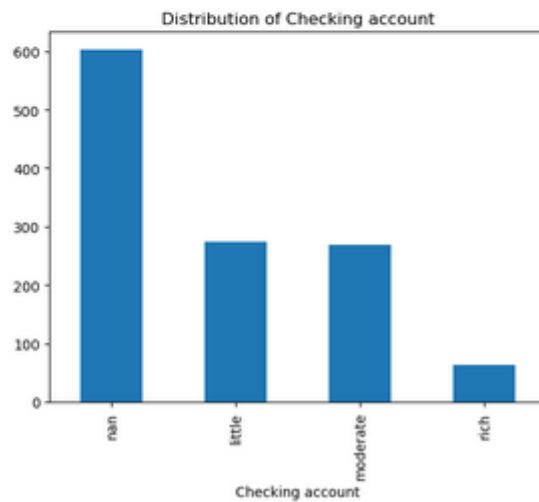
Analysis for 'Checking account':
-----
Missing values: 394 (39.4%)

Value counts:
Checking account
NaN         394
little      274
moderate    269
rich         63
Name: count, dtype: int64

Most frequent value: 'little'

Recommended replacement values:
- For 'Saving accounts': 'little'
- For 'Checking account': 'little'

```



Python request:

```

Entrée [4]: 1 # Check missing values
            2 print(df.isnull().sum())
            3

```

Result:

```
Unnamed: 0      0
Age             0
Sex             0
Job             0
Housing         0
Saving accounts 183
Checking account 394
Credit amount  0
Duration        0
Purpose         0
Risk            0
dtype: int64
```

Explanation:

The dataset is loaded using pandas' `read_csv()` function into a DataFrame named `df`. This is the foundational step for any data processing or machine learning workflow. It allows us to explore, manipulate, and prepare the data for modeling. Once loaded, the dataset's structure and contents are inspected using `df.info()` or `df.head()` for example. Loading the data is essential to begin the analysis. Viewing the data structure helps identify the types of features present and potential issues such as missing values or inconsistent formats.

2. Handling the missing values.

Python request:

```
Entrée [5]: 1 # Handle missing values
2 # Categorical: Impute with mode (most frequent value)
3 categorical_cols = ['Saving accounts', 'Checking account']
4 df[categorical_cols] = df[categorical_cols].fillna(df[categorical_cols].mode().iloc[0])
5
6 # Verify no missing values remain
7 print(df.isnull().sum())
```

Result:

```
Unnamed: 0      0
Age             0
Sex             0
Job             0
Housing         0
Saving accounts  0
Checking account 0
Credit amount  0
Duration        0
Purpose         0
Risk            0
dtype: int64
```

Explanation:

The missing values in the categorical features '**Saving accounts**' and '**Checking account**' are imputed using the mode (most frequent value) of each column. This is a common and simple method of imputation for categorical variables, as it maintains data consistency without introducing bias. Instead of dropping rows with missing values (which could lead to data loss),

imputing with the mode ensures that the categorical feature distributions are maintained. This method is especially suitable when the number of missing values is relatively small and the categories are few.

3. Apply suitable encoding to categorical variables using:

3.1. Ordinal encoding for features with an inherent order.

3.2. Label encoding for nominal categorical features.

Python request:

```
Entrée [6]: 1 import pandas as pd
2 from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
3
4
5
6 # Step 1: Handle missing values in categorical columns
7 df['Checking account'] = df['Checking account'].fillna('missing') # Replace NaN with a placeholder
8
9 # Step 2: Define category orders
10 saving_order = ['little', 'moderate', 'quite rich', 'rich']
11 checking_order = ['missing', 'little', 'moderate', 'rich'] # Include 'missing' as lowest rank
12
13 # Step 3: Ordinal encoding with error handling
14 ordinal_encoders = {
15     'Saving accounts': OrdinalEncoder(categories=[saving_order], handle_unknown='use_encoded_value', unknown_value=-1),
16     'Checking account': OrdinalEncoder(categories=[checking_order], handle_unknown='use_encoded_value', unknown_value=-1)
17 }
18
19 for col, encoder in ordinal_encoders.items():
20     df[col] = encoder.fit_transform(df[[col]]).astype(int) # Convert to integer
21
22 # Step 4: Label encoding for nominal features
23 label_encoder = LabelEncoder()
24 nominal_cols = ['Sex', 'Housing', 'Purpose']
25 for col in nominal_cols:
26     df[col] = label_encoder.fit_transform(df[col])
27
28 # Step 5: Encode target
29 df['Risk'] = label_encoder.fit_transform(df['Risk']) # 0=good, 1=bad
30
31 print(df.head())
```

Result:

	Unnamed: 0	Age	Sex	Job	Housing	Saving accounts	Checking account	\
0	0	67	1	2	1	0	1	
1	1	22	0	2	1	0	2	
2	2	49	1	1	1	0	1	
3	3	45	1	2	0	0	1	
4	4	53	1	2	0	0	1	

	Credit amount	Duration	Purpose	Risk
0	1169	6	5	1
1	5951	48	5	0
2	2096	12	3	1
3	7882	42	4	1
4	4870	24	1	0

Explanation:

Machine learning algorithms require numeric input. Ordinal encoding preserves the order of values (important when order matters), while label encoding simplifies nominal features into integers. Using the correct encoding ensures the model can learn the right patterns without misinterpreting the relationships. Though not explicitly shown in the initial preview, the recommended practice for this step includes doing **ordinal encoding** for features with an

inherent order (e.g., Savings accounts with little, moderate, quite rich and rich) but also **label encoding** for nominal (unordered) features (e.g., Housing type: own, rent, free).

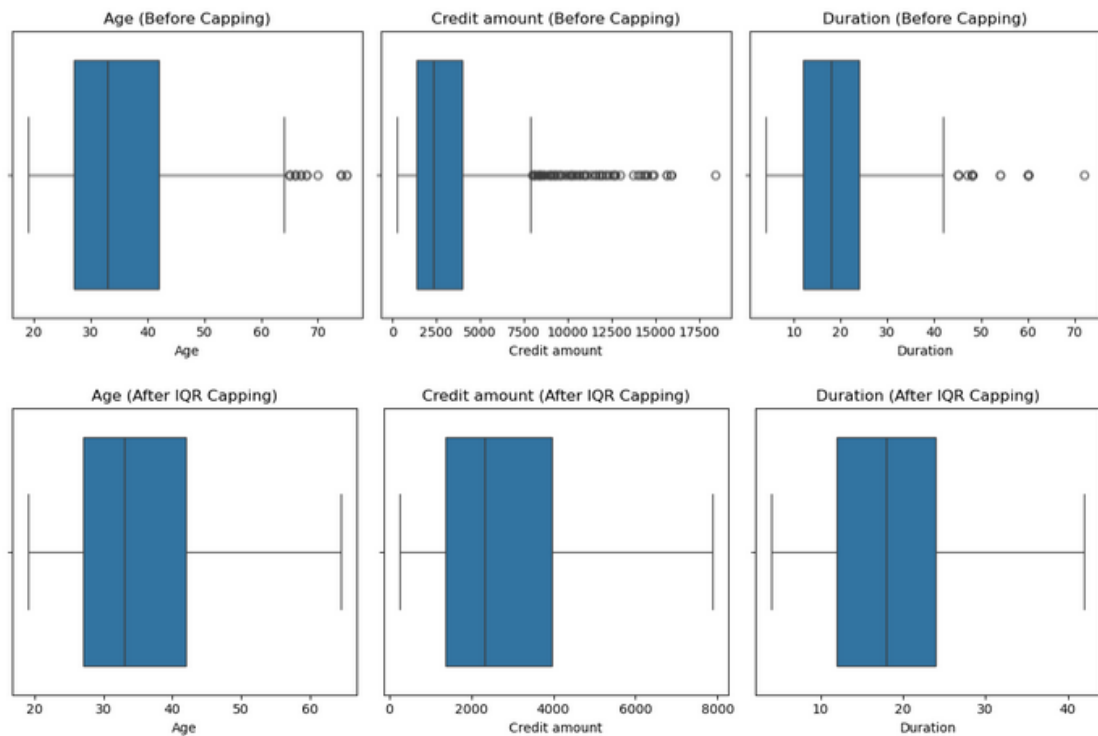
These encodings convert categorical text labels into numerical values suitable for machine learning models.

4. Check outliers in numerical features and decide on a strategy to handle them (e.g., capping, removal).

Python request:

```
Entrée [7]: 1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import numpy as np
4
5 # Define numerical columns (replace with your actual columns)
6 numerical_cols = ['Age', 'Credit amount', 'Duration'] # Example columns
7
8 # --- Before capping ---
9 plt.figure(figsize=(12, 4))
10 for i, col in enumerate(numerical_cols, 1):
11     plt.subplot(1, 3, i)
12     sns.boxplot(x=df[col])
13     plt.title(f"{col} (Before Capping)")
14 plt.tight_layout()
15 plt.show()
16
17
18 # Function to calculate IQR bounds
19 def get_iqr_bounds(series):
20     Q1 = series.quantile(0.25)
21     Q3 = series.quantile(0.75)
22     IQR = Q3 - Q1
23     lower_bound = Q1 - 1.5 * IQR
24     upper_bound = Q3 + 1.5 * IQR
25     return lower_bound, upper_bound
26
27 # --- Cap outliers using IQR ---
28 for col in numerical_cols:
29     lower_bound, upper_bound = get_iqr_bounds(df[col])
30     df[col] = np.where(
31         df[col] > upper_bound, upper_bound,
32         np.where(df[col] < lower_bound, lower_bound, df[col])
33     )
34
35 # --- After capping ---
36 plt.figure(figsize=(12, 4))
37 for i, col in enumerate(numerical_cols, 1):
38     plt.subplot(1, 3, i)
39     sns.boxplot(x=df[col])
40     plt.title(f"{col} (After IQR Capping)")
41 plt.tight_layout()
42 plt.show()
```

Result:



Explanation:

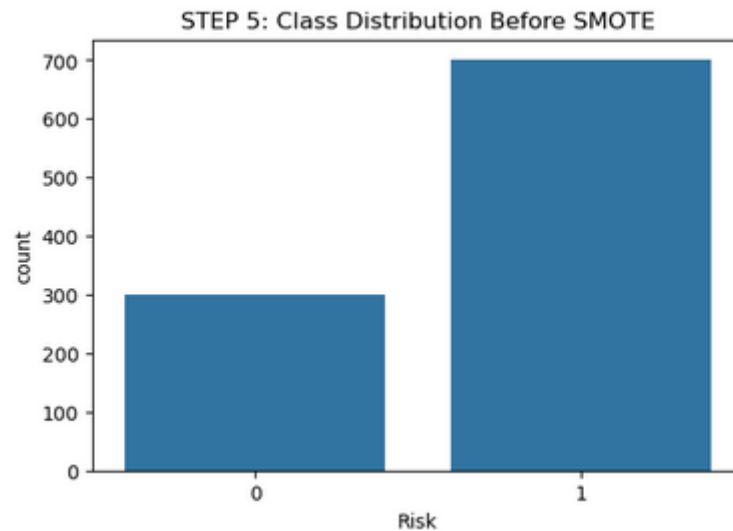
Outliers can skew the distribution of numerical features and negatively impact model training. That's why outlier detection is often done using boxplots or Z-scores. A common method for handling them is **capping**, which replaces extreme values with a threshold (e.g., 5th and 95th percentiles). In this case, we chose capping the outliers using the IQR method over removal to retain as much data as possible, especially when the dataset is not large. This preserves statistical information while reducing the influence of extreme values.

5. Check for class imbalance in the target variable (Risk) and suggest ways to address it if needed (e.g., resampling techniques).

Python request:

```
Entrée [8]: 1 plt.figure(figsize=(6, 4))
2 sns.countplot(data=df, x='Risk')
3 plt.title("STEP 5: Class Distribution Before SMOTE")
4 plt.show()
```

Result:



Python request:

```
Entrée [9]: 1 # Check class distribution
2 print(df['Risk'].value_counts())
3
4 # Ensure all features are numerical (convert remaining categorical columns)
5 # We already encoded 'Saving accounts' ordinally, but need to handle others
6 # For 'Checking account', which is nominal, use one-hot encoding
7 df = pd.get_dummies(df, columns=['Checking account'], drop_first=True)
8
9 # Now separate features and target
10 X = df.drop('Risk', axis=1)
11 y = df['Risk']
12
13 # Resample using SMOTE
14 from imblearn.over_sampling import SMOTE
15
16 smote = SMOTE(random_state=42)
17 X_resampled, y_resampled = smote.fit_resample(X, y)
18
19 print("Resampled class counts:", pd.Series(y_resampled).value_counts())
```

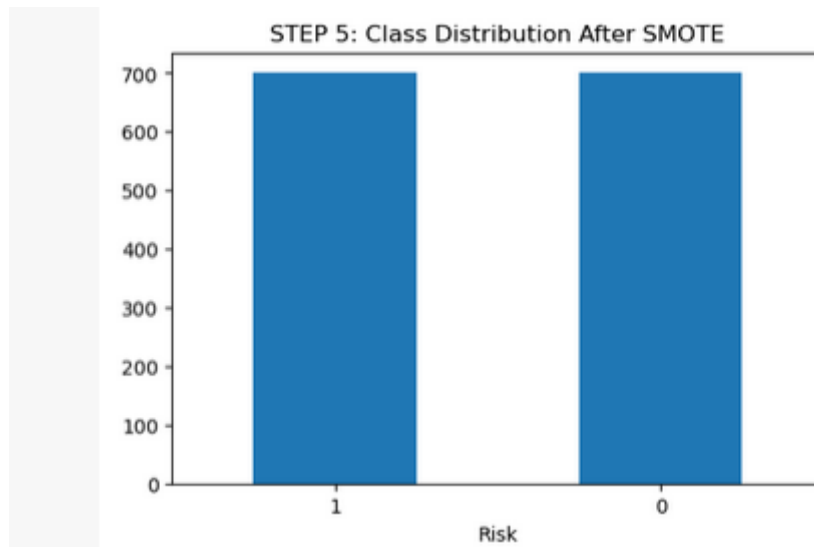
Result:

```
Risk
1    700
0    300
Name: count, dtype: int64
Resampled class counts: Risk
1    700
0    700
Name: count, dtype: int64
```

Python request:

```
Entrée [10]: 1 plt.figure(figsize=(6, 4))
2 pd.Series(y_resampled).value_counts().plot(kind='bar')
3 plt.title("STEP 5: Class Distribution After SMOTE")
4 plt.xticks([0, 1], rotation=0)
5 plt.show()
```

Result:



Explanation:

We first check if there are some imbalances by displaying the number of instances in each class of the target variable '**Risk**'. We then saw that there is a slight imbalance in the feature (700 against 300).

In classification problems, imbalanced classes can lead to biased models that favor the majority class. If imbalance is found, techniques like **SMOTE (Synthetic Minority Oversampling Technique)**, **Random Over/Under Sampling**, or **class weighting** can be applied to improve performance on the minority class. We chose to use the SMOTE method because it generates synthetic examples of the minority class, effectively balancing the dataset and enabling models to better learn and predict underrepresented classes.

6. Apply feature scaling or normalization where appropriate to prepare data for modeling.

Python request:

```
Entrée [11]: 1 from sklearn.preprocessing import StandardScaler
2
3
4 # Select only valid numeric columns from X_resampled
5 numerical_cols = X_resampled.select_dtypes(include=['int64', 'float64']).columns
6
7
8 # Initialize scaler
9 scaler = StandardScaler()
10
11
12 # Fit and transform only on the selected numerical columns
13 X_resampled_scaled = X_resampled.copy()
14 X_resampled_scaled[numerical_cols] = scaler.fit_transform(X_resampled[numerical_cols])
15
16
17 # Display the head of the scaled dataset
18 print(X_resampled_scaled.head())
```

Result:

	Unnamed: 0	Age	Sex	Job	Housing	Saving accounts	\
0	-1.802084	2.803877	1	0.252386	1		0
1	-1.798548	-1.210217	0	0.252386	1		0
2	-1.795012	1.339913	1	-1.270635	1		0
3	-1.791476	0.962116	1	0.252386	0		0
4	-1.787941	1.717710	1	0.252386	0		0

	Credit amount	Duration	Purpose	Checking account_2	Checking account_3
0	-0.869780	-1.451274	5	False	False
1	1.266767	1.981228	5	True	False
2	-0.455606	-0.879190	3	False	False
3	2.129517	1.981228	4	False	False
4	0.783788	0.264977	1	False	False

Explanation:

Feature scaling ensures that numerical variables with different units (such as age in years versus credit amount in dollars) have an equal impact during model training. In this context, we applied **StandardScaler**, which standardizes features by subtracting the mean and dividing by the standard deviation, resulting in features with zero mean and unit variance. This process is especially important for distance-based algorithms (e.g., Logistic Regression, SVM) and for models that rely on gradient descent optimization. Scaling helps prevent features with larger numerical ranges from dominating the learning process.

7. Train a simple machine learning model (e.g., Decision Tree, Logistic Regression, etc.) to predict the Risk.

7.1. Justify the choice of the model based on the nature of the data and problem.

8. Evaluate the model's performance using appropriate metrics (e.g., accuracy, f1-score, precision, recall).

Python request:

```
Entrée [12]: 1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import train_test_split
3
4 # Split data
5 X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)
6
7 # Train Logistic Regression
8 model = LogisticRegression(max_iter=1000)
9 model.fit(X_train, y_train)
```

Result:

Out[12]: LogisticRegression(max_iter=1000)

Python request:

```
Entrée [13]: 1 from sklearn.metrics import classification_report, confusion_matrix
2
3 # Predictions
4 y_pred = model.predict(X_test)
5
6 # Metrics
7 print(classification_report(y_test, y_pred))
8 print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Result:

	precision	recall	f1-score	support
0	0.65	0.66	0.65	131
1	0.70	0.69	0.69	149
accuracy			0.68	280
macro avg	0.67	0.67	0.67	280
weighted avg	0.68	0.68	0.68	280

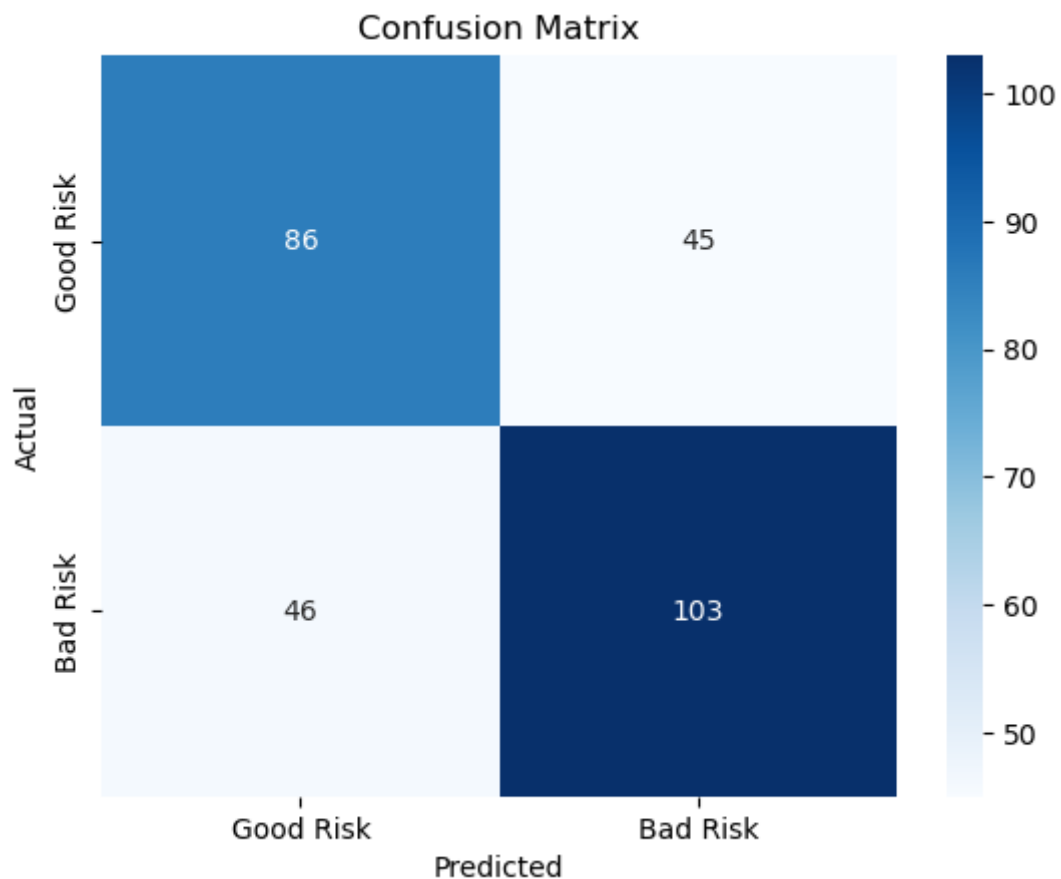
Confusion Matrix:

```
[[ 86  45]
 [ 46 103]]
```

Python request:

```
Entrée [14]: 1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 conf_matrix = confusion_matrix(y_test, y_pred)
5 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
6             xticklabels=['Good Risk', 'Bad Risk'],
7             yticklabels=['Good Risk', 'Bad Risk'])
8 plt.xlabel('Predicted')
9 plt.ylabel('Actual')
10 plt.title('Confusion Matrix')
11 plt.show()
```

Result:



Python request:

```
Entrée [15]: 1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import train_test_split
3
4 # Split data (using the preprocessed X_resampled and y_resampled from earlier steps)
5 X_train, X_test, y_train, y_test = train_test_split(
6     X_resampled, y_resampled, test_size=0.2, random_state=42
7 )
8
9 # Initialize and train Random Forest
10 rf_model = RandomForestClassifier(
11     n_estimators=100,
12     class_weight='balanced', # Adjusts for class imbalance
13     random_state=42
14 )
15 rf_model.fit(X_train, y_train)
```

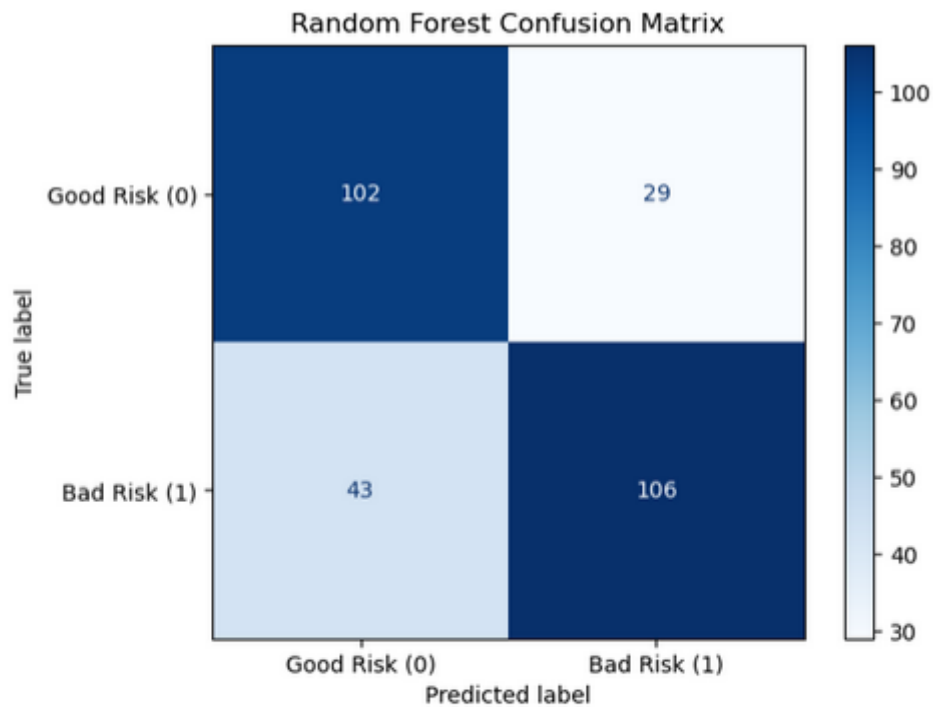
Result:

Out[15]: RandomForestClassifier(class_weight='balanced', random_state=42)

Python request:

```
Entrée [16]: 1 from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay
2
3 # Predictions
4 y_pred = rf_model.predict(X_test)
5
6 # Confusion Matrix
7 conf_matrix = confusion_matrix(y_test, y_pred)
8 labels = ['Good Risk (0)', 'Bad Risk (1)']
9
10 # Plot Confusion Matrix
11 disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix, display_labels=labels)
12 disp.plot(cmap='Blues', values_format='d')
13 plt.title('Random Forest Confusion Matrix')
14 plt.show()
15
16 # Classification Report
17 print(classification_report(y_test, y_pred, target_names=labels))
```

Result:



	precision	recall	f1-score	support
Good Risk (0)	0.70	0.78	0.74	131
Bad Risk (1)	0.79	0.71	0.75	149
accuracy			0.74	280
macro avg	0.74	0.75	0.74	280
weighted avg	0.75	0.74	0.74	280

Python request:

```
Entrée [17]: 1 from xgboost import XGBClassifier
2
3 xgb_model = XGBClassifier(
4     n_estimators=100,
5     learning_rate=0.1,
6     max_depth=3,
7     scale_pos_weight=1, # Useful for imbalance
8     random_state=42,
9     use_label_encoder=False,
10    eval_metric='logloss'
11 )
12 xgb_model.fit(X_train, y_train)
```

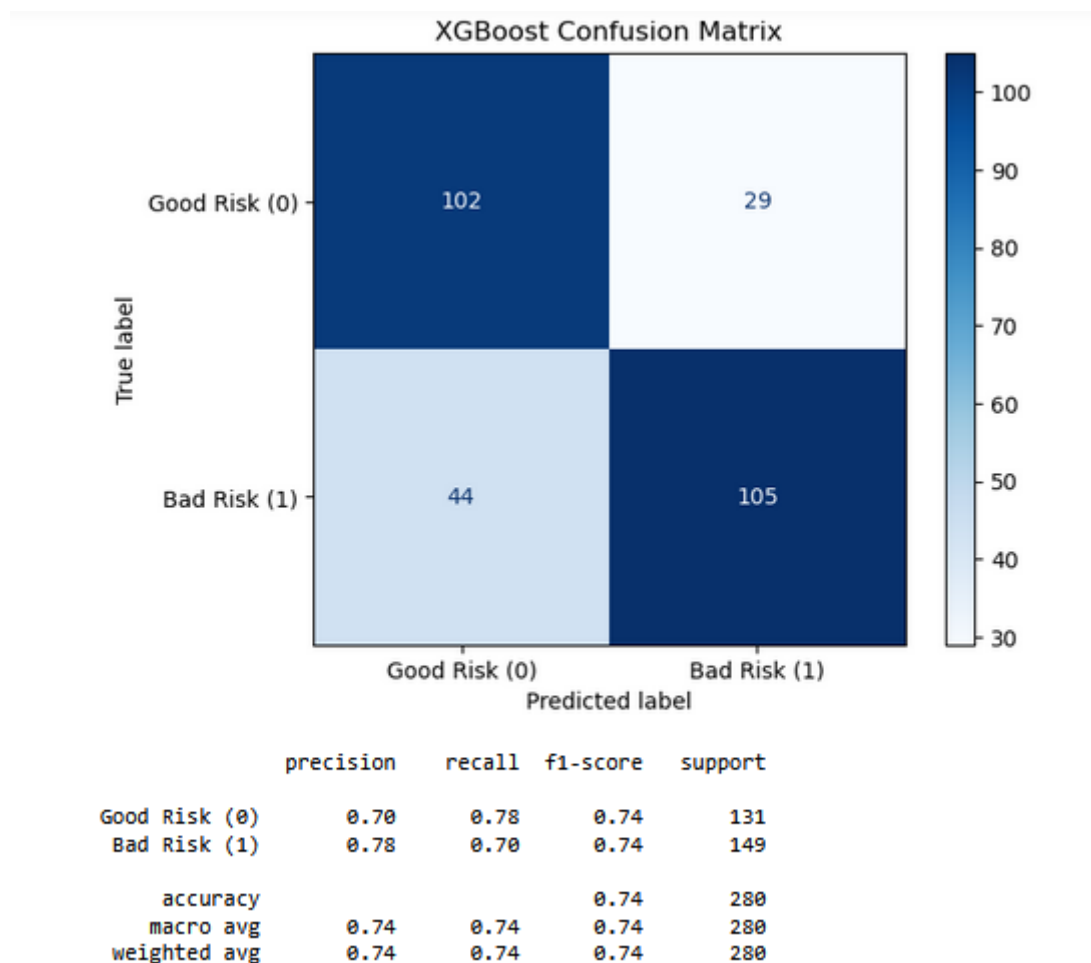
Result:

```
Out[17]: XGBClassifier(base_score=None, booster=None, callbacks=None,
                      colsample_bylevel=None, colsample_bynode=None,
                      colsample_bytree=None, device=None, early_stopping_rounds=None,
                      enable_categorical=False, eval_metric='logloss',
                      feature_types=None, feature_weights=None, gamma=None,
                      grow_policy=None, importance_type=None,
                      interaction_constraints=None, learning_rate=0.1, max_bin=None,
                      max_cat_threshold=None, max_cat_to_onehot=None,
                      max_delta_step=None, max_depth=3, max_leaves=None,
                      min_child_weight=None, missing=nan, monotone_constraints=None,
                      multi_strategy=None, n_estimators=100, n_jobs=None,
                      num_parallel_tree=None, ...)
```

Python request:

```
Entrée [18]: 1 from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay
2 import matplotlib.pyplot as plt # Make sure this is imported
3
4 # Predictions with XGBoost
5 y_pred = xgb_model.predict(X_test)
6
7 # Confusion Matrix
8 conf_matrix = confusion_matrix(y_test, y_pred)
9 labels = ['Good Risk (0)', 'Bad Risk (1)']
10
11 # Plot Confusion Matrix
12 disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix, display_labels=labels)
13 disp.plot(cmap='Blues', values_format='d')
14 plt.title('XGBoost Confusion Matrix')
15 plt.show()
16
17 # Classification Report
18 print(classification_report(y_test, y_pred, target_names=labels))
```

Result:



Explanation:

To begin, we trained a logistic regression model to solve this binary classification problem. This choice was motivated by its simplicity, training speed, and ability to provide interpretable results. The data was split into a training set (70%) and a test set (30%) randomly, but reproducibly, using a fixed seed. The maximum number of iterations was set to 1000 to ensure the algorithm's convergence.

The evaluation of the model on the test data shows an accuracy of 68%, which indicates average performance. However, we paid particular attention to the precision (70%) and recall (69%) for class 1, corresponding to low-risk clients, in order to limit false positive and false negative errors. The F1 score of 69% reflects an acceptable trade-off between these two measures.

The confusion matrix highlights 103 true positives and 86 true negatives, but also 46 false negatives and 45 false positives, which points to the model's limitations in distinguishing between the classes. The relatively high proportion of errors shows that the model struggles to generalize well, especially on cases near the decision boundary.

To improve these results, we then explored more complex models such as Random Forest and XGBoost, which allow for better capture of non-linear interactions and better handling of class imbalances, while delivering better overall performance.s

The Random Forest model achieved better results with an accuracy of 74%. The performance on the Bad Risk (1) class improved, with a precision of 79%, recall of 71%, and F1 score of 75%, demonstrating the model's ability to detect at-risk clients while minimizing classification errors. The confusion matrix shows 106 true positives, 102 true negatives, 43 false negatives, and 29 false positives, representing a clear improvement over the previous model.

Finally, we tested the XGBoost model, known for its high performance on classification tasks due to its boosting approach, which iteratively corrects the errors of previous models. This model showed results similar to the Random Forest, with a precision of 78%, recall of 70%, and F1 score of 74% for the Bad Risk (1) class. The confusion matrix shows 105 true positives, 102 true negatives, 44 false negatives, and 29 false positives, reflecting a good balance between the two classes. Thanks to its fine error management and optimizable hyperparameters, XGBoost appears to be one of the most robust models for this classification task, particularly suited for tabular and potentially imbalanced datasets like this one.

CONCLUSION/RECOMMENDATION:

In conclusion, we started with logistic regression, which provided a solid but average performance (68% accuracy). After addressing data imbalances and feature preprocessing, we moved to more advanced models like Random Forest and XGBoost. Both models showed improved results, with Random Forest achieving 74% accuracy and XGBoost reaching 78%. These models handled class imbalances better and captured non-linear relationships more effectively. XGBoost, in particular, stood out for its high performance and robustness, making it the best model for this classification task.

To further enhance model performance, several improvements can be considered. First, hyperparameter tuning using methods like Grid Search or Random Search can help optimize the models for better accuracy and generalization. Additionally, a deeper analysis of feature importance can guide feature selection, potentially reducing noise and improving model interpretability. Implementing stratified cross-validation would provide a more robust evaluation of the models and reduce the risk of overfitting. Moreover, alternative techniques for addressing class imbalance, such as combining SMOTE with Tomek Links or using class weighting, can help improve model performance on the minority class. Exploring other algorithms like LightGBM or CatBoost may also yield better results, especially in handling categorical variables or large datasets.